



The Foundation

11%

Module 1: QUICK START (Get Buildi... ^

1.1 What You Need (5 Min Setup) ✓

1.2 Your First Folder ✓

1.3 How to Structure Any Prompt

Module 2: WHY THIS WORKS (The ... v

Module 3: FOLDER ARCHITECTURE ... v

Module 4: CLAUDE CODE (Getting H... v

1.2 Your First Folder ✓

What You'll Get From This

A working folder that changes how Claude responds to you. Three files. Five minutes. You will see the difference immediately.

The Lesson

Right now, most people open Claude, type a question, and start over every time. Every conversation begins from zero. You spend half the message explaining who you are, what you are working on, and what you need. Then you hit the token limit, start a new chat, and do it all again.

project is, and what good work looks like. Claude reads them and its responses change. You will notice it in the first message.

This is the quick version. We are getting you a win right now. If you want to understand why this works (context windows, tokens, routing, the full

architecture), that is all shown and explained in **Section 3**. You do not need any of that to start. You just need three files.

Step 1: Make the Folder

Open your code editor (VS Code or Cursor from Lesson 1.1). Create a new folder. Name it after whatever you are working on.

```
my-first-workspace/
```

Step 2: Create Three Files

Inside that folder, create three files. Use `.md` (*markdown*) if you can. Plain `.txt` works too.

File 1: **CLAUDE.md**

This tells Claude who it is working for and how to behave.

```
# Identity
```



```
## Rules
```

- Write in plain, clear language
- Ask clarifying questions before making assumptions

- When you are unsure, say so

Replace the brackets with your info. Keep it short.

File 2: [CONTEXT.md](#)

This tells Claude what you are working on right now.

```
# Current Project

## What we are building
[Describe your project in 2-3 sentences]

## What good looks like
[What does a successful output look like?]

## What to avoid
[Common mistakes or things you do not want]
```

File 3: [REFERENCES.md](#)



```
# References

## Examples of good work
[Paste an example or describe what you liked about it]
```

Relevant links

[URLs, docs, resources for this project]

Notes

[Anything else Claude should know]

That is it. Three files.

Step 3: Point Claude At It

If you are using Claude Code (terminal): Navigate to the folder, type `claude`. It reads the files automatically.

```
cd my-first-workspace
claude
```

Ask it something about your project. Notice how different the response is.



and start a conversation inside that Project. Claude references them in every message.

- **Copy and paste:** Copy the contents of all three files and paste them at the top of your first message. Less elegant. Works fine.

Step 4: See The Difference

Ask Claude something related to your project. Compare what you get now versus what you would have gotten cold.

The response should feel more specific, more relevant, more like talking to someone who already knows what you are working on. Because now it does.

What just happened?

You gave Claude three things it never has in a normal chat: who you are, what the project is, and what good looks like. That is the whole idea. The files carry context so you do not have to re-explain it every time.

You can edit these files whenever your project changes. You can add more files when you need Claude to know more. You can create separate folders for separate projects. The system grows with you.

This is the starting point. There is a full architecture underneath this (three layers, routing tables, naming conventions, production pipelines) and it is all covered in [Section 3](#) when you are ready for it. But everything in [Section 3](#) builds on what you just did. These three files are the foundation.



- **Ready for the full system?** Section 3 is the complete Folder Architecture method. [3.1: The Full Walkthrough](#) is a 23-minute video and written companion covering the three-layer routing system, naming conventions, customization for your use case, and the research behind it. Everything you just set up here gets explained at a deeper level there.

- **Want to understand why this works?** Section 2 covers the ideas underneath. [2.3: How a 1953 Word Game Explains AI Memory](#) explains context windows and why structured input changes the output. [2.2: One Line of Python Triggers 12,000 Lines of Code](#) traces the full computing stack and makes the case that building structure around AI is the same engineering humans have done for 200 years.
- **Premium and VIP members:** The full Workspace Blueprint template (the production version of what you just built) is in [The Vault](#).

Next Up: [1.3 How to Structure Any Prompt](#)

You have a folder. Now you need to know how to talk to Claude inside it. The folder gives Claude memory. The prompt gives Claude direction. They work together.



Matthew Creamer

7

Mar 14 • Lesson Check-Ins



Foundations 1.2 Check-In

You built your first folder. Vote below, then drop a screenshot in the comments so we can see what you came up with.

Poll

1362 members have voted



552



499



[New comment 16h ago](#)